

University of Asia Pacific

Department of Computer Science and Engineering

CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 05

Experiment Title: Design and Simulation of an 8-bit Arithmetic Logic Unit (ALU) Supporting Addition and Subtraction Using NAND-Based Structural VHDL in Xilinx ISE

Submitted by:

Name: MD Rawnak Shalahen Fahim

Student ID: 22201063

Section: C1

Submitted to:

Md. Rasheduzzaman
Lecturer
Department of Computer Science and Engineering

Date of Submission: 10/04/2026

1. Title of the Experiment

Design and Simulation of an 8-bit Arithmetic Logic Unit (ALU) Supporting Addition and Subtraction Using NAND-Based Structural VHDL in Xilinx ISE

2. Objectives

The main objectives of this experiment are:

1. To understand the design and operation of basic digital logic gates.
2. To implement a **NAND gate using VHDL dataflow modeling**.
3. To construct the **NOT, AND, OR, and XOR gates** using NAND gate instances only.
4. To design a **Half Adder** using XOR and AND gates.
5. To design a **Full Adder** using two Half Adders and an OR gate, and extend it to an **8-bit Full Adder**.
6. To design a **Full Subtractor** using two XOR gates, two AND gates, two NOT gates, and one OR gate, and extend it to an **8-bit Subtractor**.
7. To design an **8-bit Multiplexer (MUX)** for selecting between two 8-bit data paths.
8. To design an **8-bit ALU** capable of performing addition and subtraction.
9. To understand **structural modeling** in VHDL and hierarchical digital circuit design.
10. To simulate all designed circuits using **Xilinx ISE (ISim)** and verify the outputs against the expected truth tables and calculations.

3. Software and Tools Used

The following tools were used for this experiment:

- **Xilinx ISE 14.7**
- **VHDL**
- **ISim Simulator**
- Computer system for design and simulation

4. Introduction

An Arithmetic Logic Unit (ALU) is the part of a processor that performs arithmetic and logical operations on binary data. It is one of the most important building blocks of a CPU. In this experiment, an **8-bit ALU** was designed that can perform two operations: **addition** and **subtraction**. The operation to be performed is chosen by a select input.

The ALU was built using a **hierarchical structural approach**. First, a NAND gate was implemented using VHDL dataflow modeling. Since NAND is a universal gate, it was used to build the NOT, AND, OR, and XOR gates. A Half Adder was then built using XOR and AND gates, and a Full Adder was built using two Half Adders and an OR gate. Eight Full Adders were connected together to form an 8-bit Full Adder.

For subtraction, a Full Subtractor was designed using two XOR gates, two AND gates, two NOT gates, and one OR gate. Eight Full Subtractors were connected to form an 8-bit Subtractor. An 8-bit MUX was then used to select the output of either the adder or the subtractor, and all these blocks were combined to form the final ALU.

Therefore, the overall design follows the hierarchy:

NAND Gate
↓
Basic Logic Gates (NOT, AND, OR, XOR)
↓
Half Adder
↓
Full Adder / Full Subtractor
↓
8-bit Full Adder and 8-bit Subtractor
↓
8-bit MUX
↓
8-bit ALU

5. NAND Gate Design

The NAND gate is a universal logic gate. A two-input NAND gate produces the inverse of the AND operation.

The Boolean expression is:

$$Y = \overline{A \cdot B}$$

Truth Table

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

The NAND gate was implemented using VHDL **dataflow modeling**. This NAND gate is used as the only primitive building block for every other circuit in this experiment.

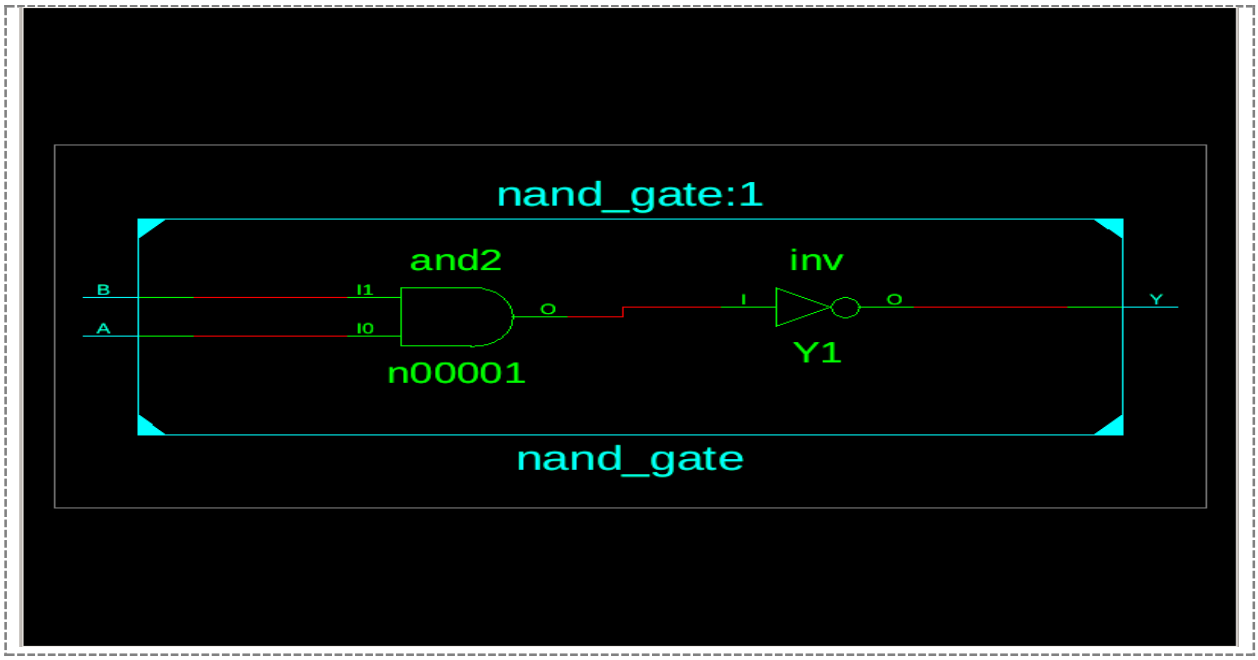


Figure 1: RTL schematic of the NAND gate.

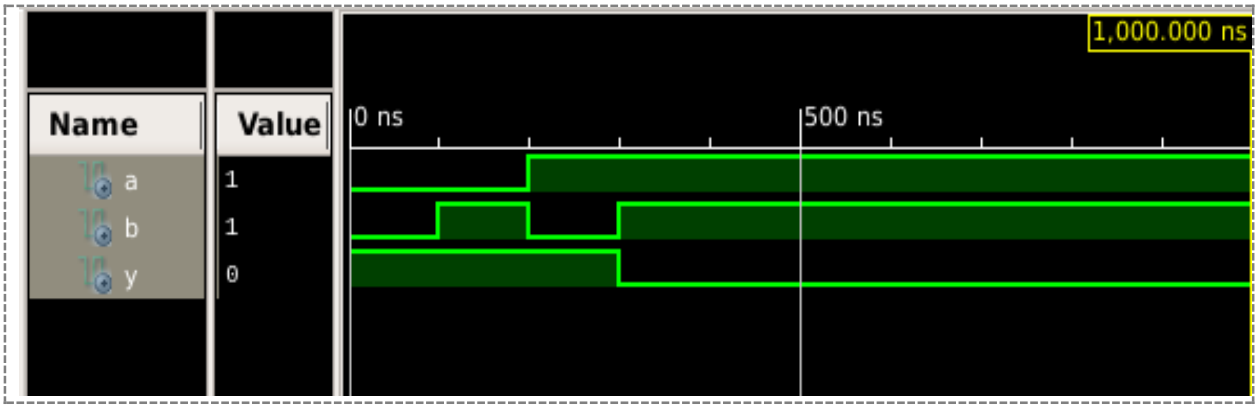


Figure 2: Simulation waveform of the NAND gate.

6. Construction of Basic Gates Using NAND Gates

Since NAND is a universal gate, every other logic gate can be constructed using NAND gates only. The following gates were designed using NAND gate instances:

- NOT
- AND
- OR
- XOR

These gates were implemented using **structural VHDL modeling** by instantiating the NAND gate component.

6.1 NOT Gate Using NAND Gate

A NOT gate is obtained by connecting both inputs of a NAND gate together.

Y = A NAND A
Therefore:

$$Y = \overline{A}$$

Truth Table

A	Y
0	1
1	0

Number of NAND gates
1 NAND gate

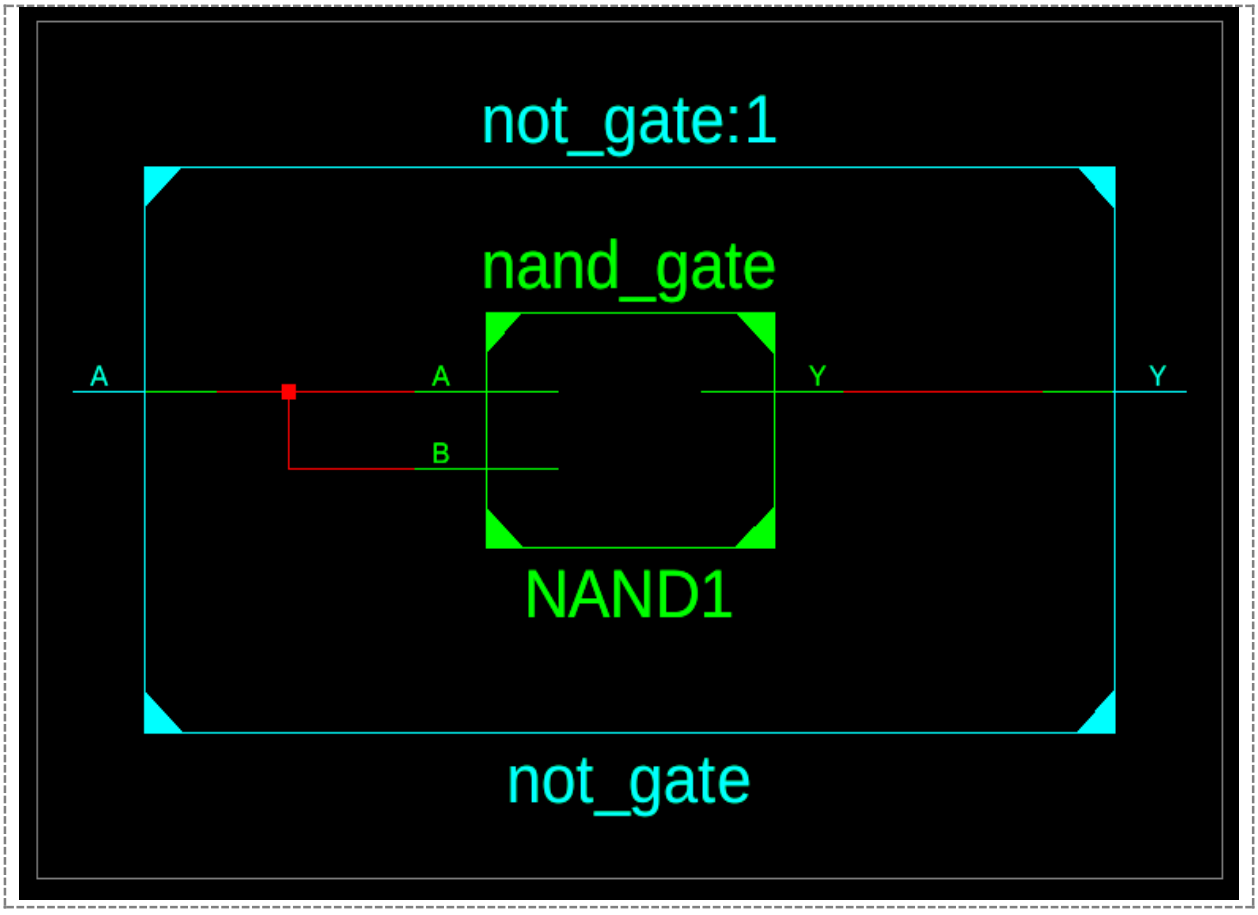


Figure 3: RTL schematic of the NAND-based NOT gate.

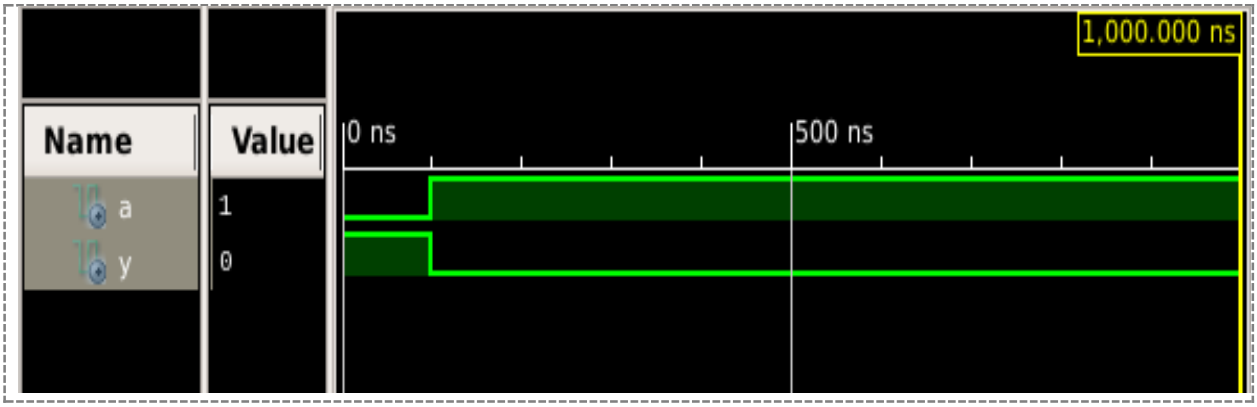


Figure 4: Simulation waveform of the NOT gate.

6.2 AND Gate Using NAND Gates

An AND gate can be constructed using two NAND gates.

First:
N1 = A NAND B
Then:
Y = N1 NAND N1
Therefore:

$$Y = A \cdot B$$

Number of NAND gates
2 NAND gates

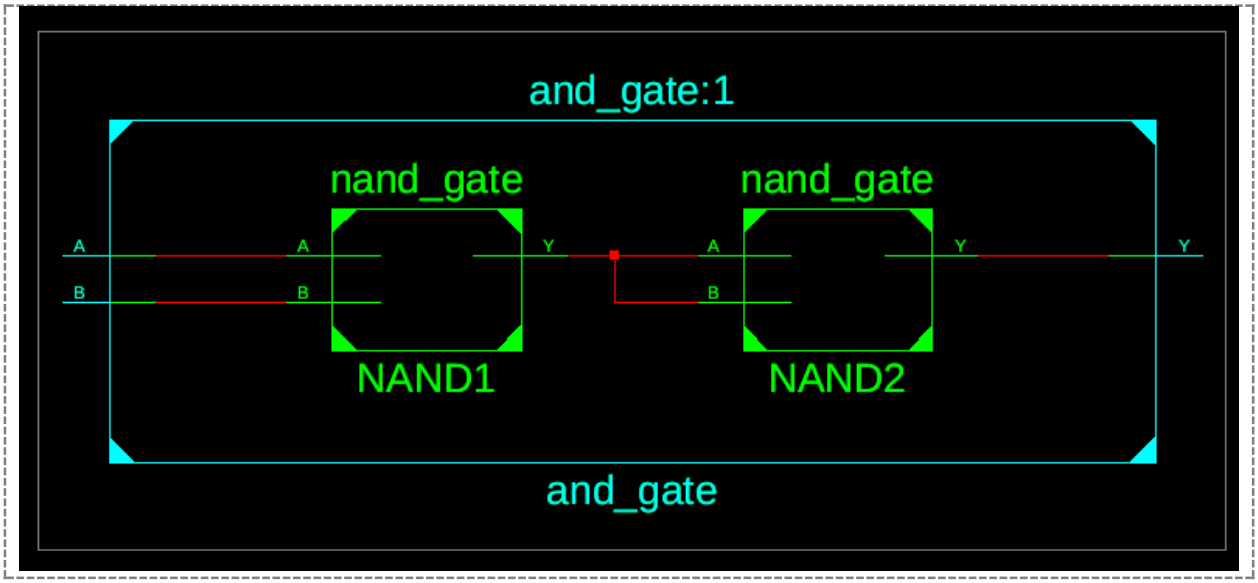


Figure 5: RTL schematic of the NAND-based AND gate.

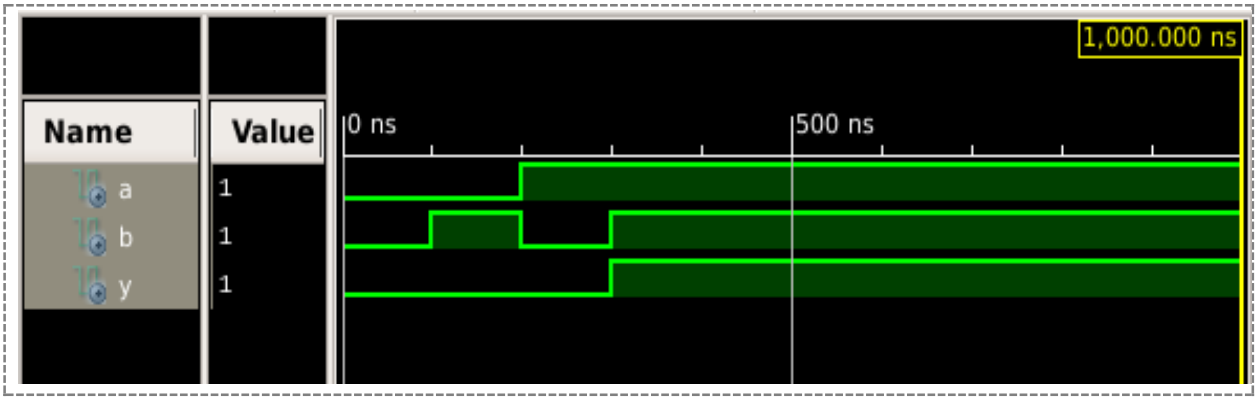


Figure 6: Simulation waveform of the AND gate.

6.3 OR Gate Using NAND Gates

Using De Morgan's theorem:

$$A + B = \overline{\overline{A} \cdot \overline{B}}$$

Therefore:
N1 = A NAND A
N2 = B NAND B
Y = N1 NAND N2

Thus, an OR gate requires **3 NAND gates**.

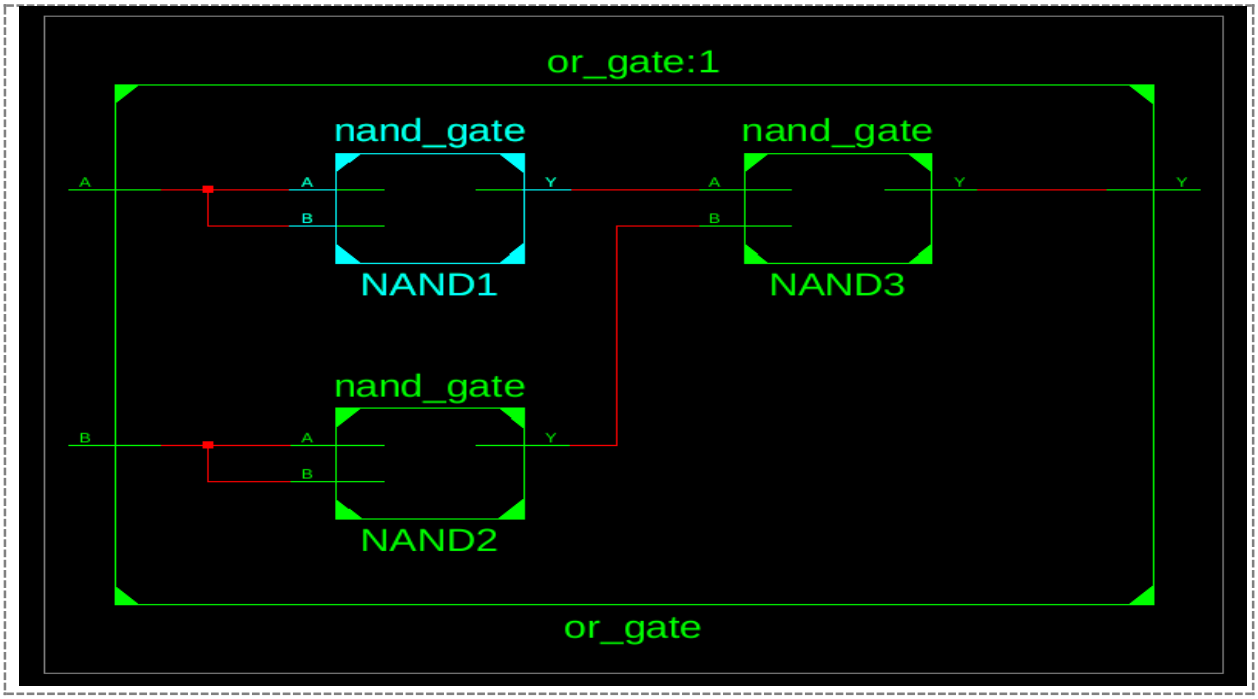


Figure 7: RTL schematic of the NAND-based OR gate.

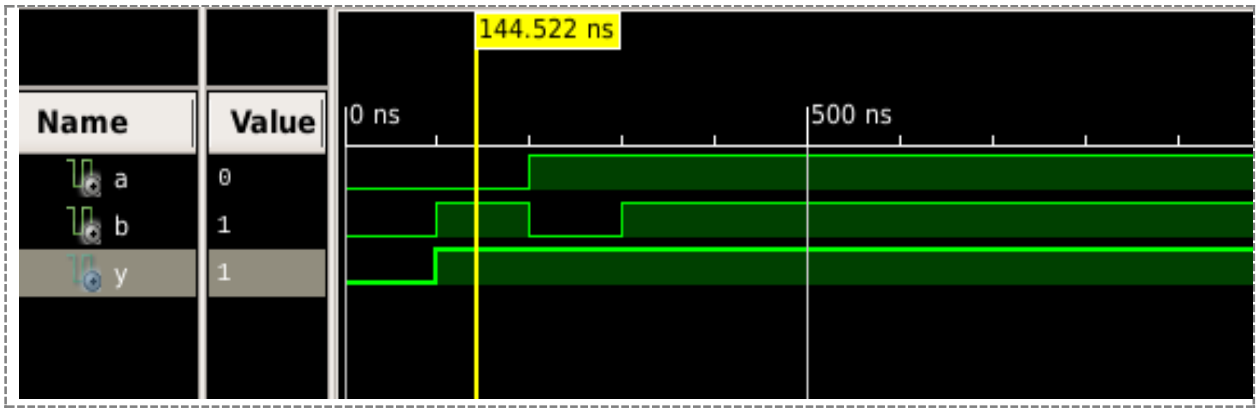


Figure 8: Simulation waveform of the OR gate.

6.4 XOR Gate Using NAND Gates

The XOR gate produces logic 1 when the two inputs are different.

The NAND-only implementation uses four NAND gates:
N1 = A NAND B
N2 = A NAND N1
N3 = B NAND N1
Y = N2 NAND N3

Therefore:

$$Y = A \oplus B$$

Truth Table

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

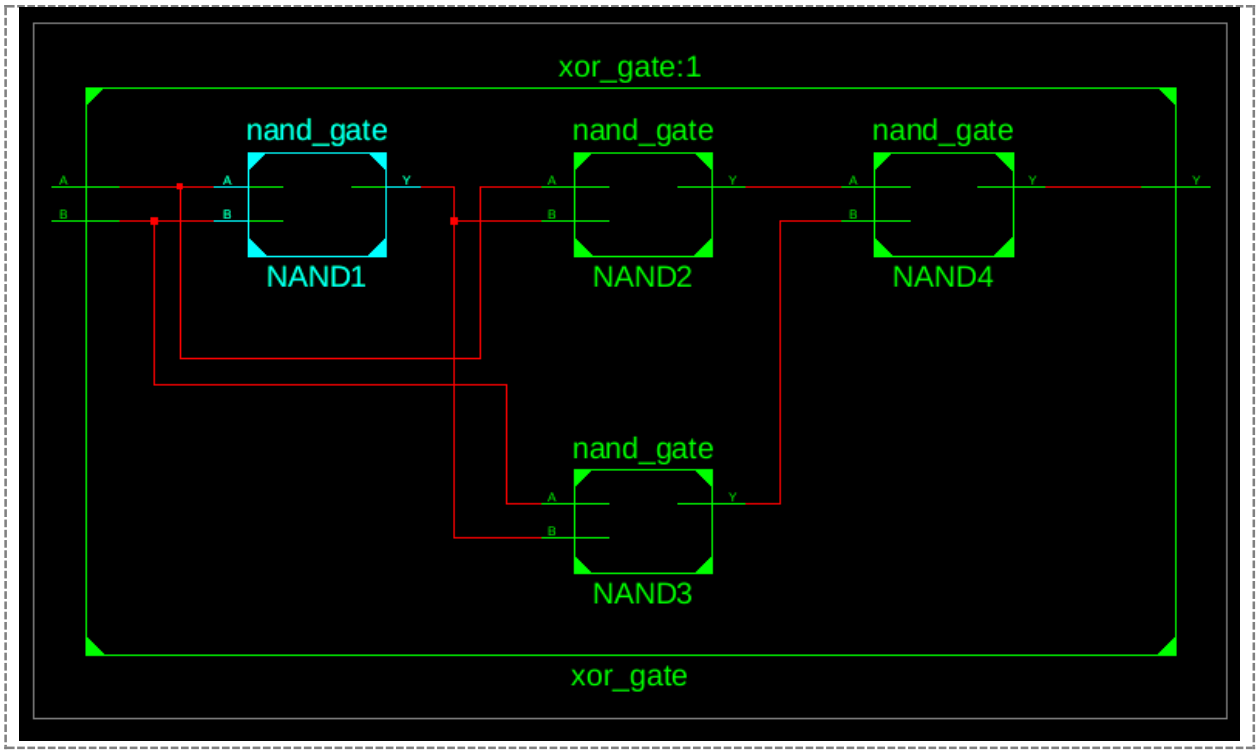


Figure 9: RTL schematic of the NAND-based XOR gate.

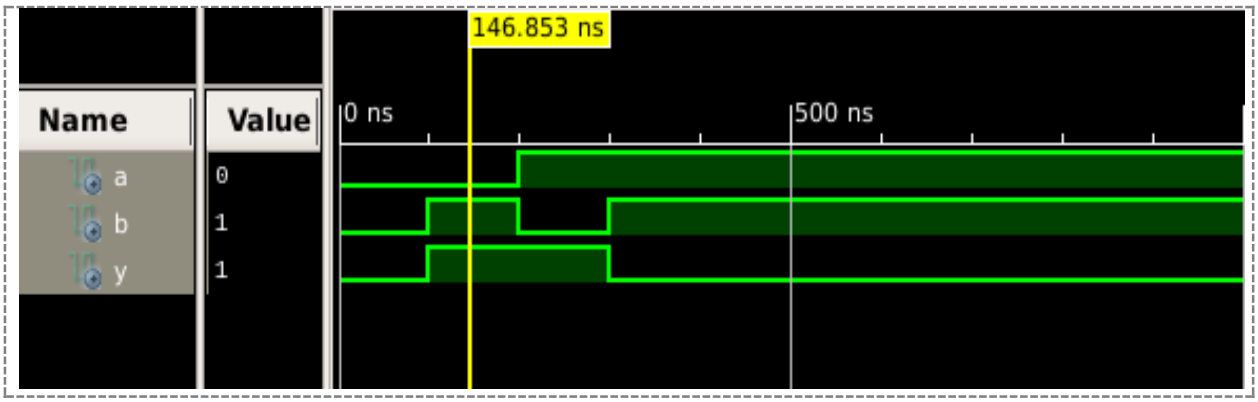


Figure 10: Simulation waveform of the XOR gate.

6.5 Summary of NAND-Based Gates

The table below summarizes the number of NAND gates required to construct each basic gate.

Gate	NAND Implementation	No. of NAND Gates
NOT	A NAND A	1
AND	(A NAND B) NAND (A NAND B)	2
OR	(A NAND A) NAND (B NAND B)	3
XOR	Four-NAND structure (Section 6.4)	4

7. Half Adder Design

A Half Adder is a combinational circuit that adds two one-bit binary numbers.

It has:

Inputs

- A
- B

Outputs

- Sum
- Carry

The Boolean expressions are:

$$Sum = A \oplus B$$

$$Carry = A \cdot B$$

The Half Adder was implemented structurally using the previously designed NAND-based XOR and AND components.

Block Diagram

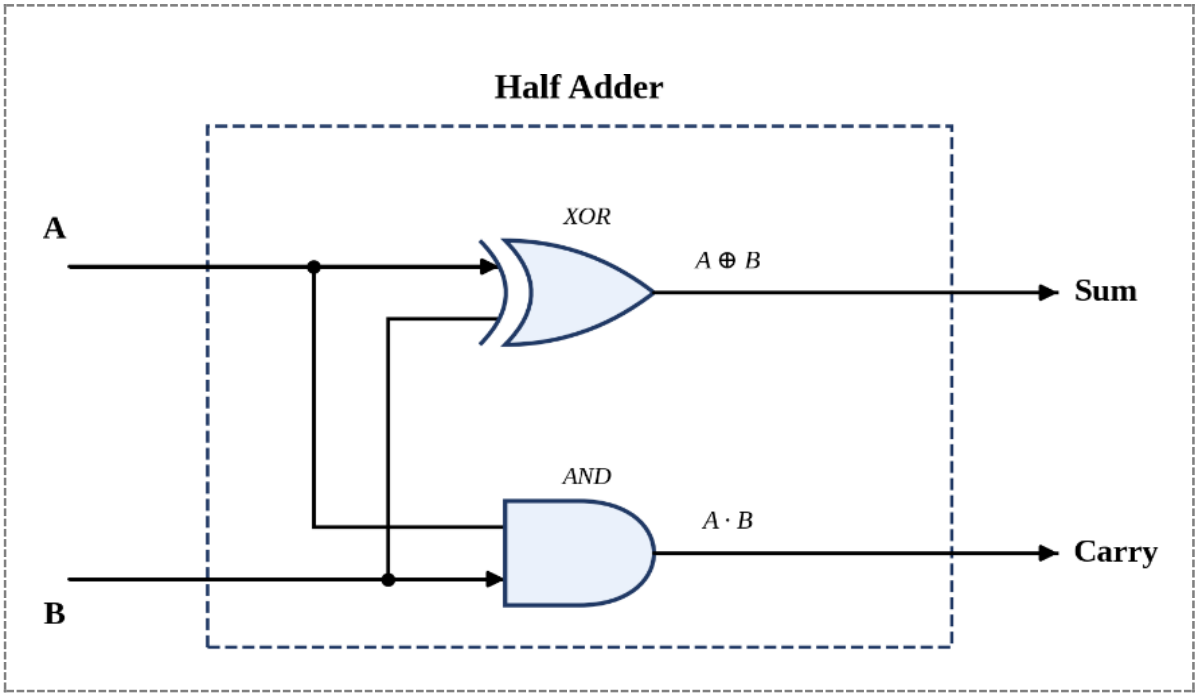


Figure 11: Block diagram of the Half Adder.

Half Adder Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

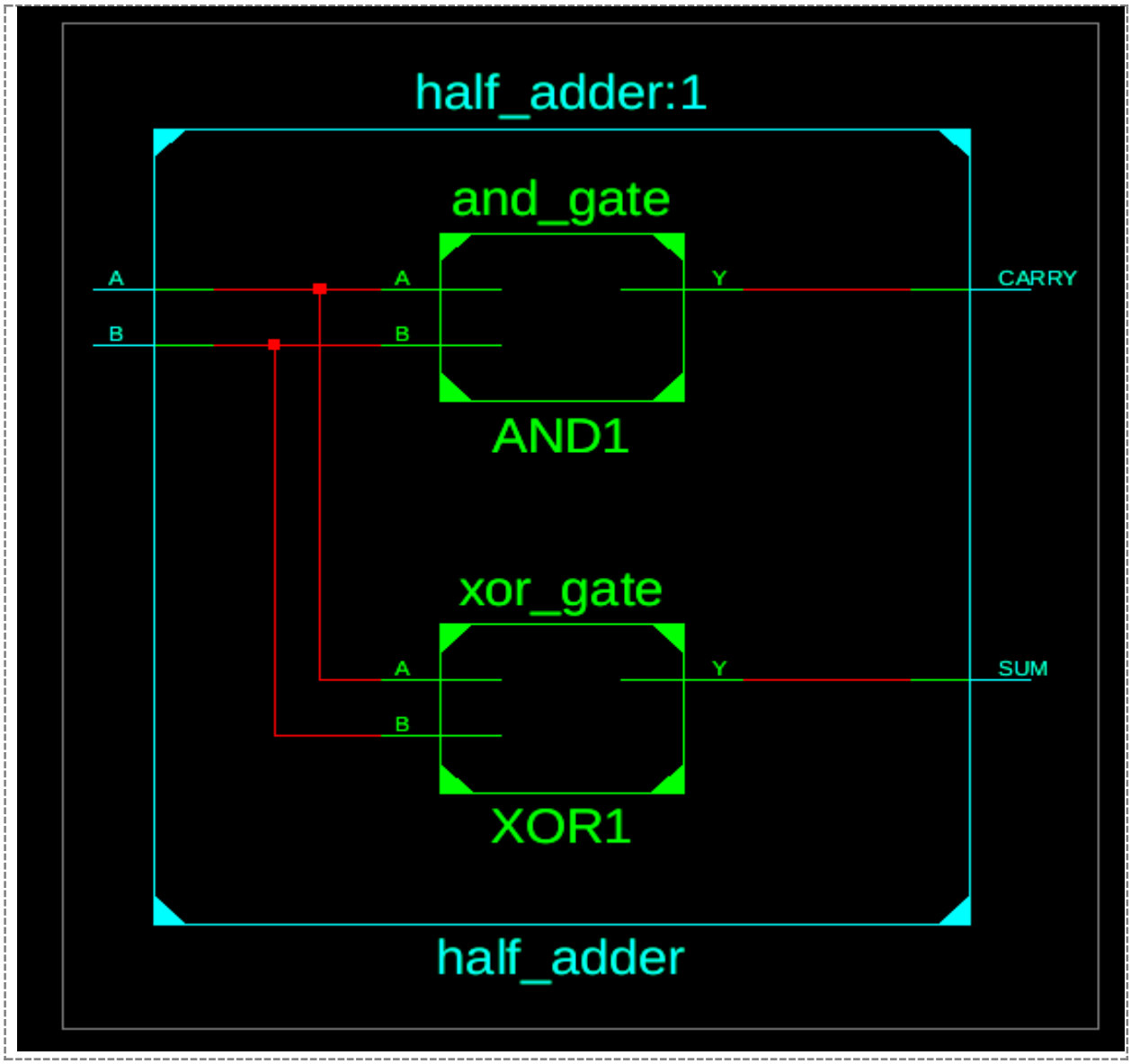


Figure 12: RTL schematic of the Half Adder.

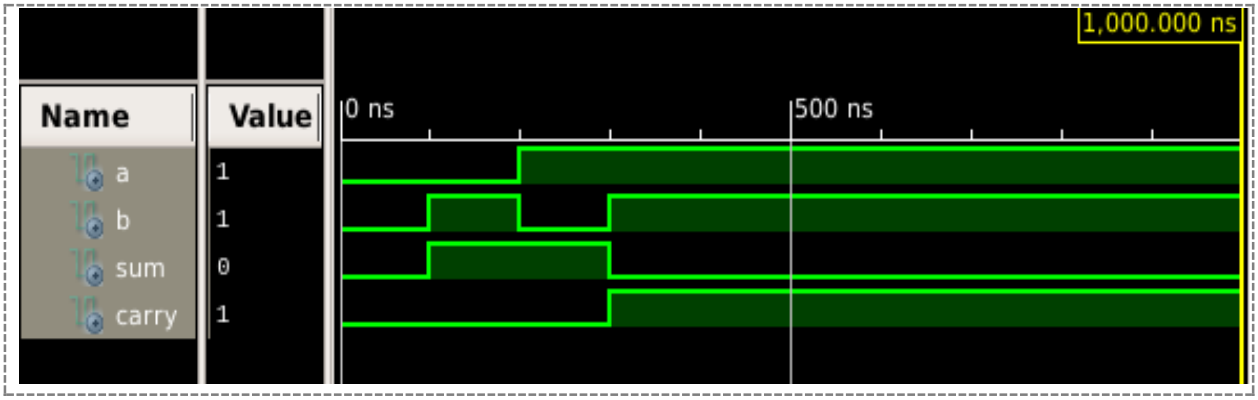


Figure 13: Simulation waveform of the Half Adder.

8. Full Adder Design

A Full Adder adds three one-bit binary values.

Inputs

- A
- B
- Cin

Outputs

- Sum
- Cout

A Full Adder can be constructed using:

- Two Half Adders
- One OR gate

The first Half Adder adds A and B. The second Half Adder adds the intermediate sum to Cin. Finally, the two carry outputs are passed through an OR gate.

8.1 Full Adder Block Diagram

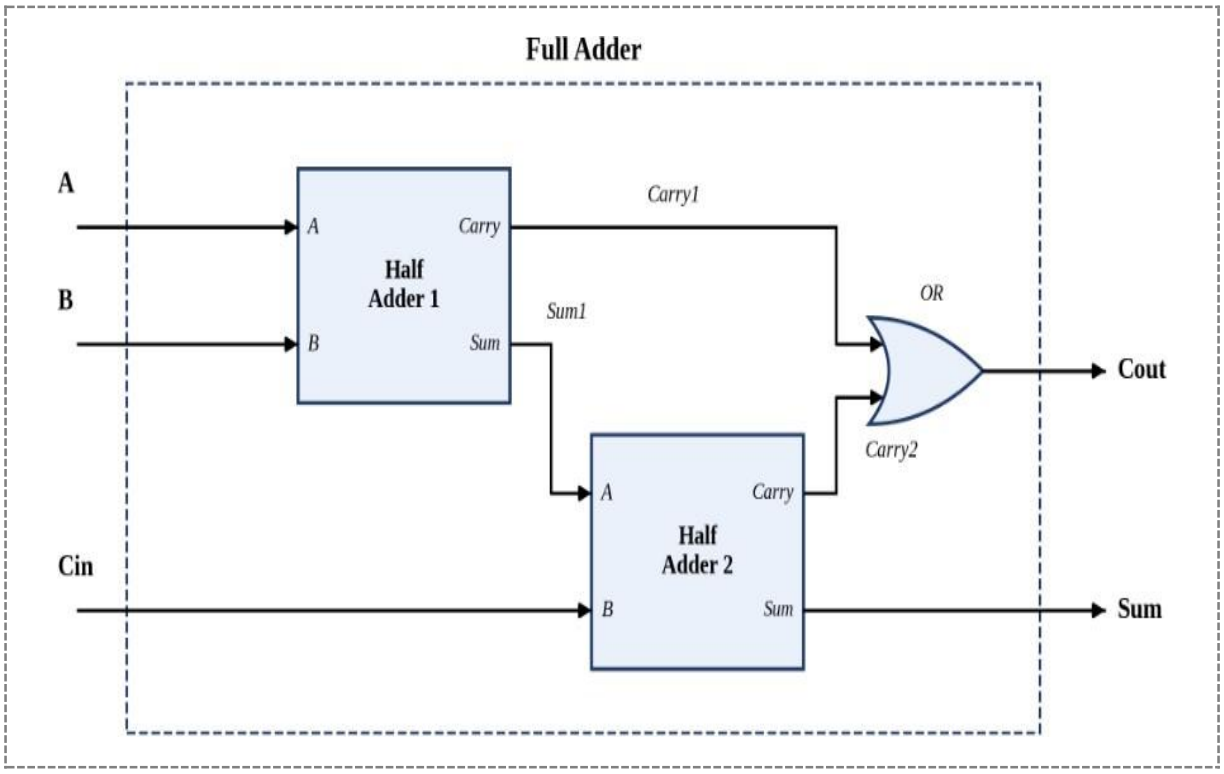


Figure 14: Block diagram of the Full Adder using two Half Adders and an OR gate.

8.2 Full Adder Logic

The first Half Adder generates:

$$Sum1 = A \oplus B$$

$$Carry1 = A \cdot B$$

The second Half Adder generates:

$$Sum = Sum1 \oplus Cin$$

$$Carry2 = Sum1 \cdot Cin$$

Finally:

$$Cout = Carry1 + Carry2$$

Therefore:

$$Sum = A \oplus B \oplus Cin$$

and:

$$Cout = AB + (A \oplus B)Cin$$

8.3 Full Adder Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

8.4 RTL Schematic and Simulation of the Full Adder

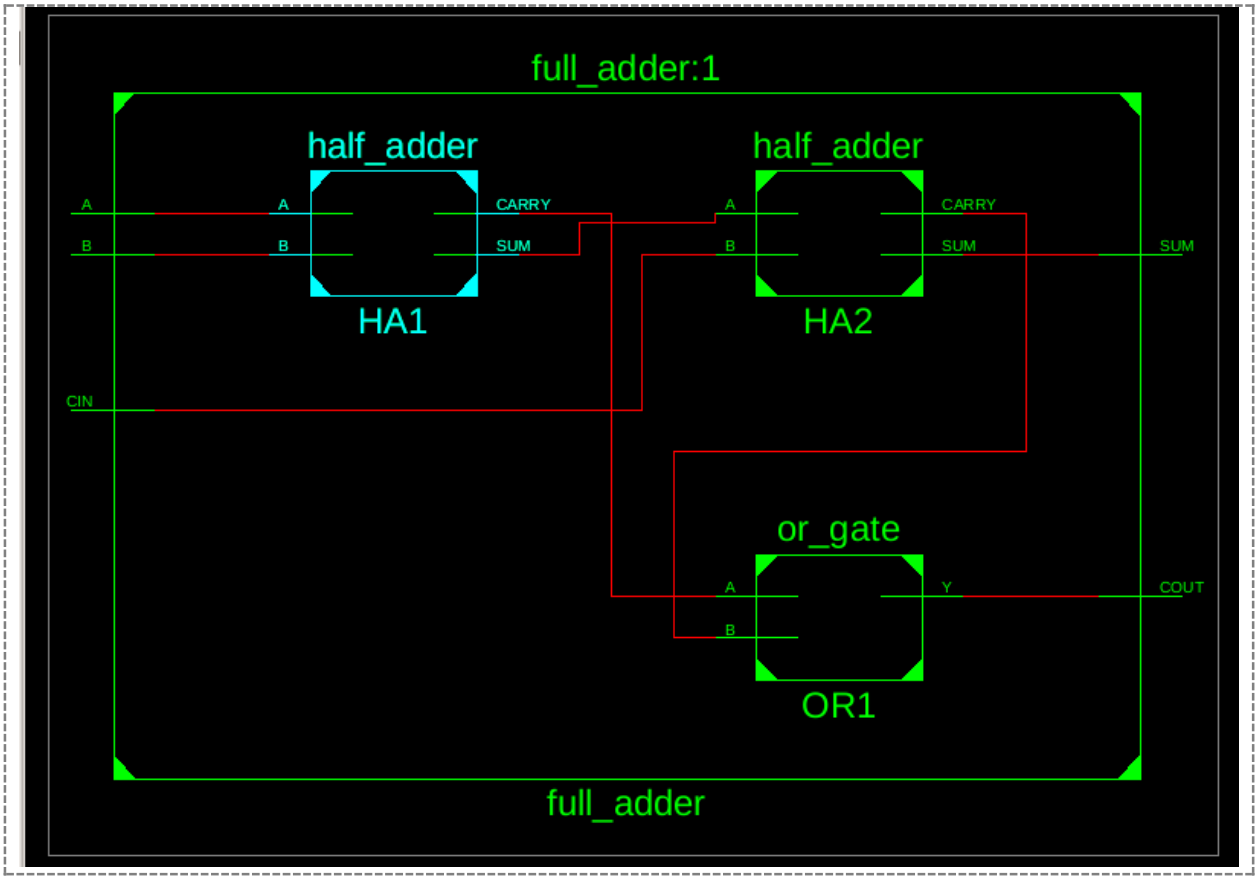


Figure 15: RTL schematic of the complete Full Adder.

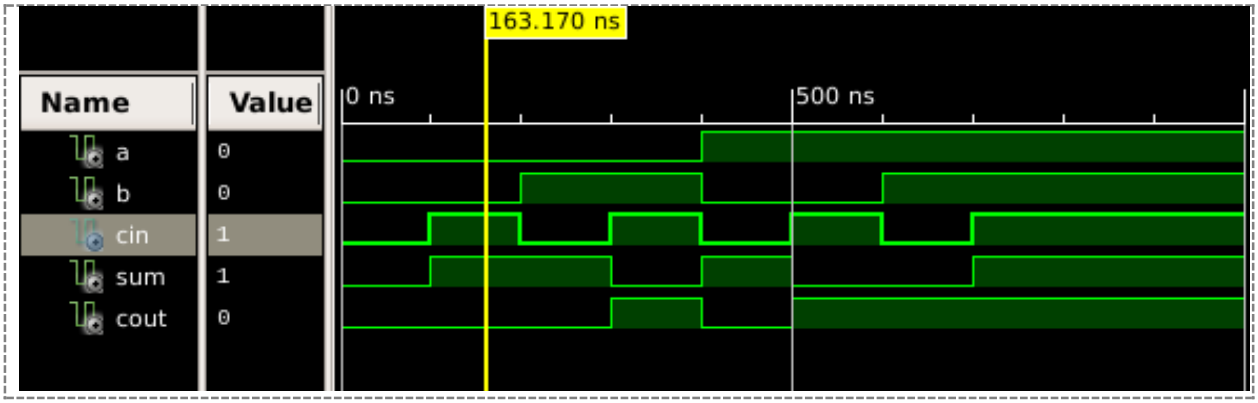


Figure 16: Simulation waveform of the Full Adder showing all eight input combinations.

9. 8-bit Full Adder Design

An 8-bit Full Adder adds two 8-bit binary numbers. It was built by connecting **eight 1-bit Full Adders in a ripple-carry arrangement**. The carry output (Cout) of each Full Adder is connected to the carry input (Cin) of the next, more significant Full Adder.

Inputs

- A[7:0] – 8-bit first operand
- B[7:0] – 8-bit second operand
- Cin – initial carry input

Outputs

- Sum[7:0] – 8-bit sum
- Cout – final carry output (set when the result exceeds 255)

For bit position i (i = 0 to 7), the equations are:

$$Sum_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i$$

where C0 = Cin and Cout = C8.

Block Diagram

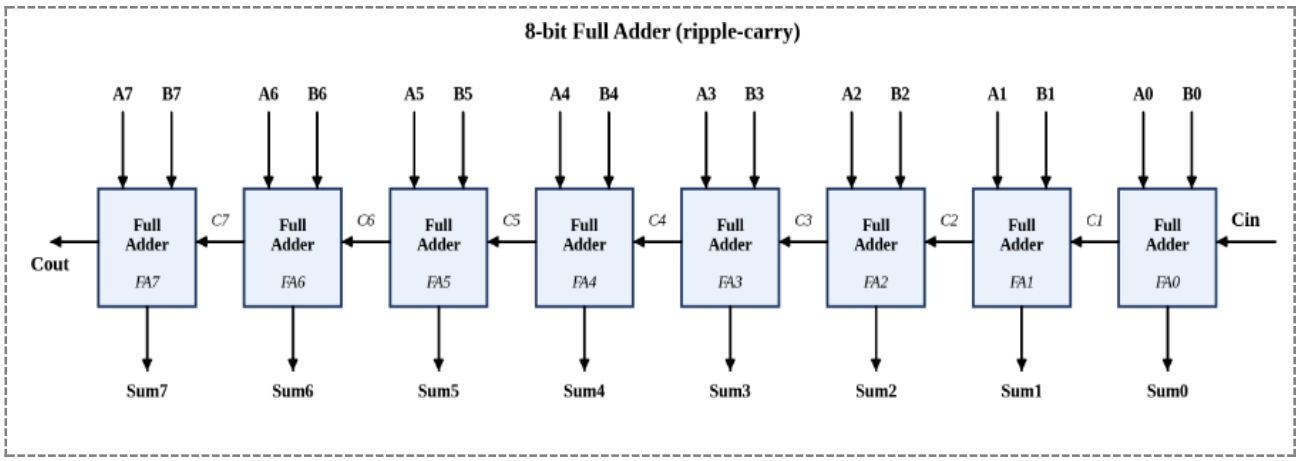


Figure 17: Block diagram of the 8-bit Full Adder (ripple-carry structure).

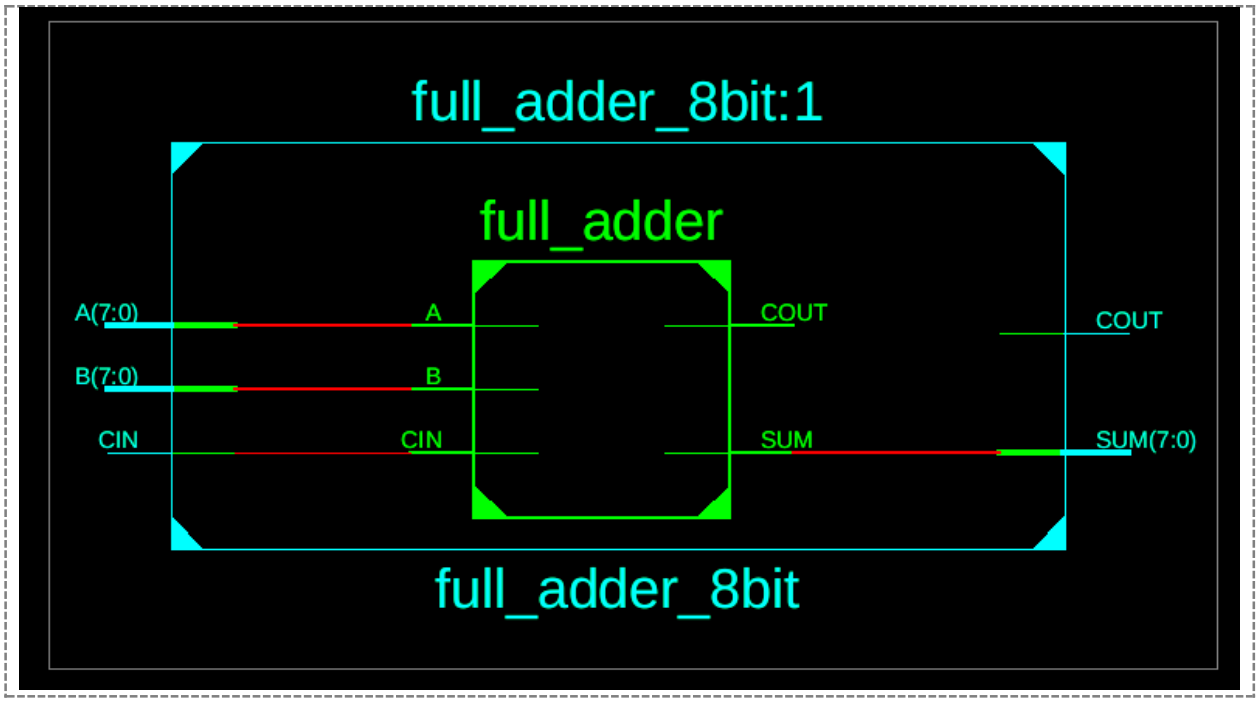


Figure 18: RTL schematic of the 8-bit Full Adder.

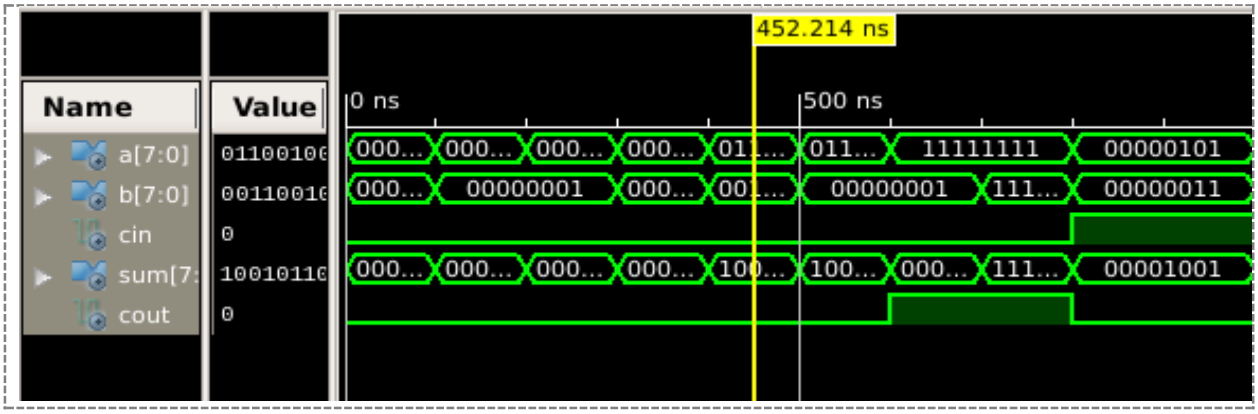


Figure 19: Simulation waveform of the 8-bit Full Adder.

10. Full Subtractor Design

A Full Subtractor subtracts one bit (B) and a borrow input (Bin) from another bit (A). It produces a difference and a borrow output.

Inputs

- A
- B
- Bin

Outputs

- Diff

- Bout

The Boolean expressions are:

$$Diff = A \oplus B \oplus Bin$$

$$Bout = \overline{A} \cdot B + \overline{A \oplus B} \cdot Bin$$

The Full Subtractor was implemented structurally using the following gates:

Gate	Quantity	Purpose
XOR	2	Generate A⊕B and the Difference output
AND	2	Generate the two borrow terms
NOT	2	Generate A' and (A⊕B)'
OR	1	Combine the two borrow terms to form Bout

The internal signals are:
X1 = A XOR B
Diff = X1 XOR Bin
N1 = NOT A
N2 = NOT X1
T1 = N1 AND B
T2 = N2 AND Bin
Bout = T1 OR T2

Full Subtractor Block Diagram

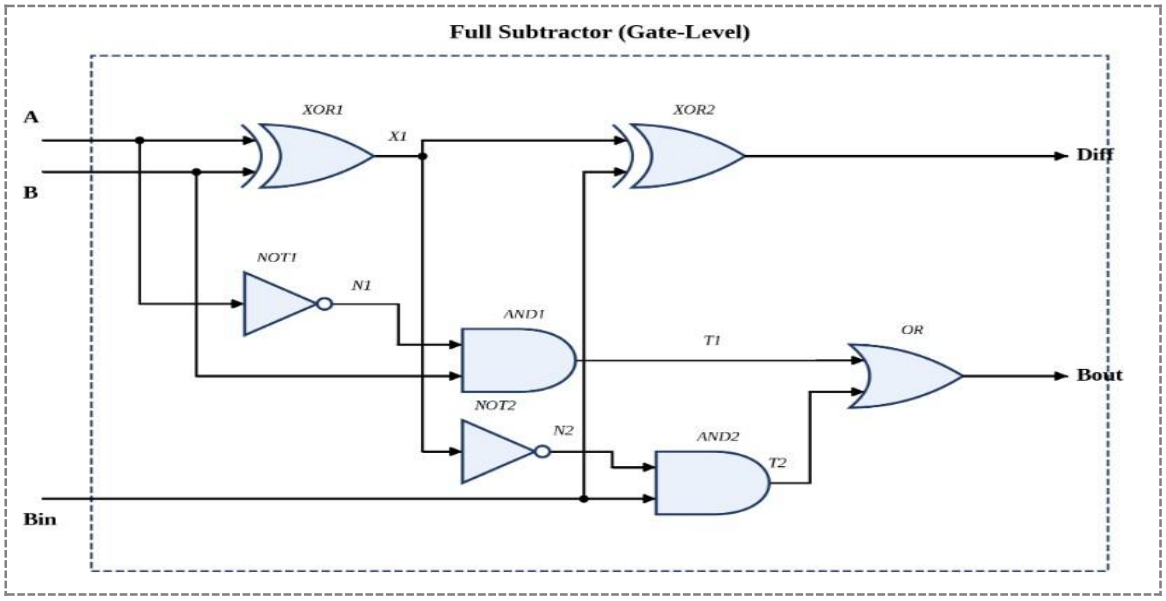


Figure 20: Gate-level block diagram of the Full Subtractor.

Full Subtractor Truth Table

A	B	Bin	Diff	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

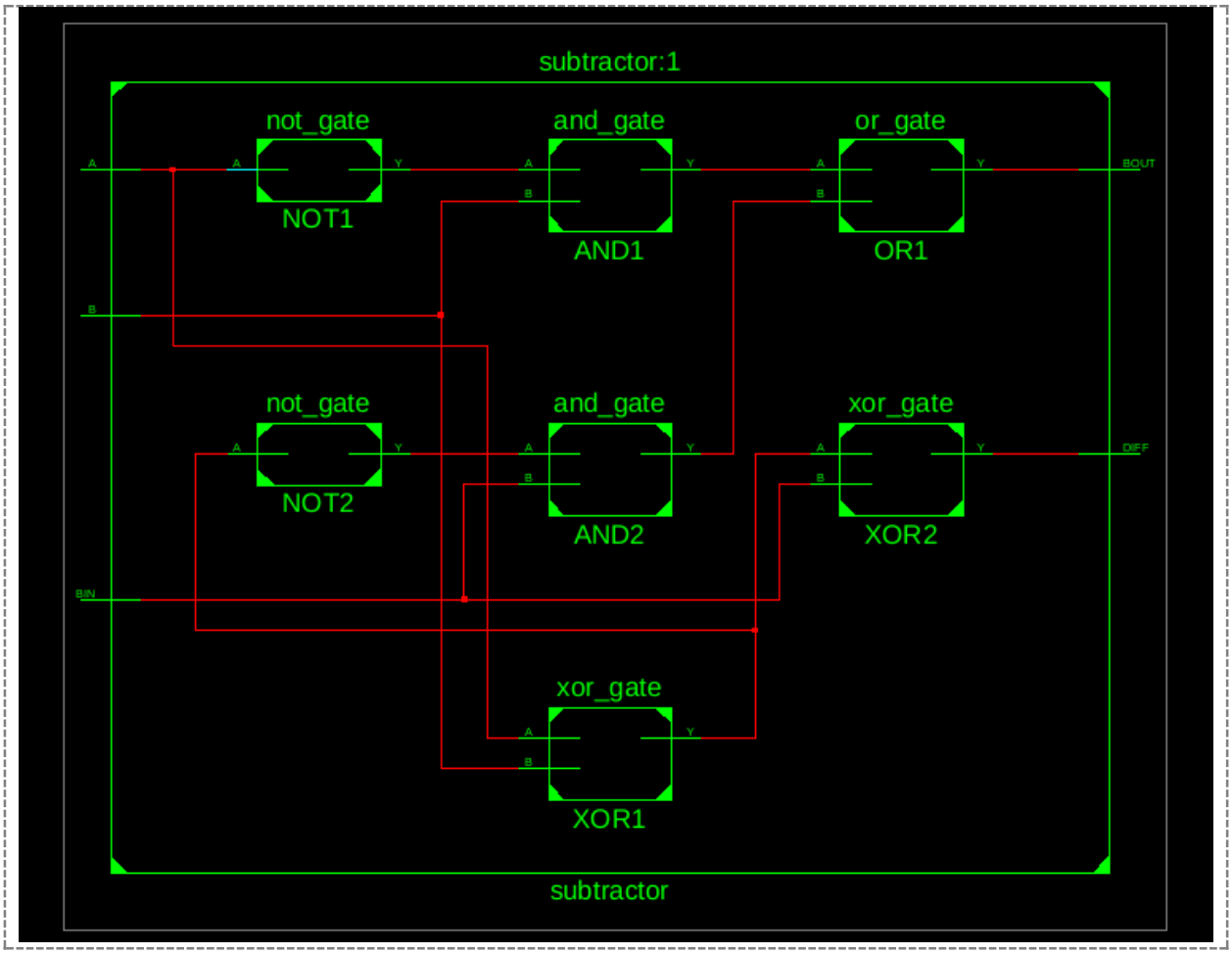


Figure 21: RTL schematic of the Full Subtractor.

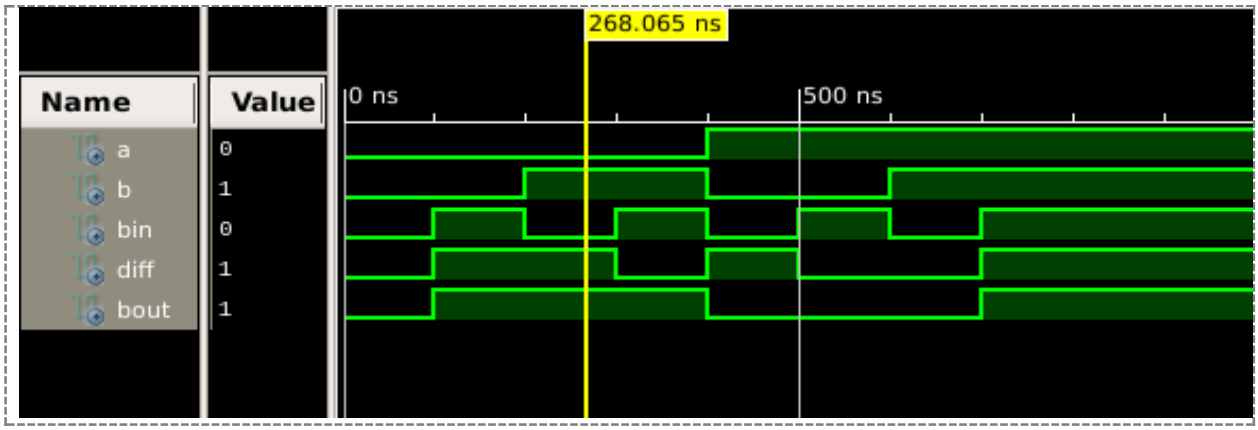


Figure 22: Simulation waveform of the Full Subtractor.

11. 8-bit Subtractor Design

An 8-bit Subtractor computes A − B for two 8-bit numbers. It was built by connecting **eight 1-bit Full Subtractors in a ripple-borrow arrangement**, where the borrow output (Bout) of each stage is connected to the borrow input (Bin) of the next stage.

Inputs

- A[7:0] – 8-bit minuend
- B[7:0] – 8-bit subtrahend
- Bin – initial borrow input

Outputs

- Diff[7:0] – 8-bit difference
- Bout – final borrow output (set when A < B)

For bit position i (i = 0 to 7):

$$Diff_i = A_i \oplus B_i \oplus Bin_i$$

$$Bin_{i+1} = \overline{A_i}B_i + \overline{A_i \oplus B_i}Bin_i$$

When A is smaller than B, the final borrow is 1 and the 8-bit difference is displayed in 2's complement form.

Block Diagram

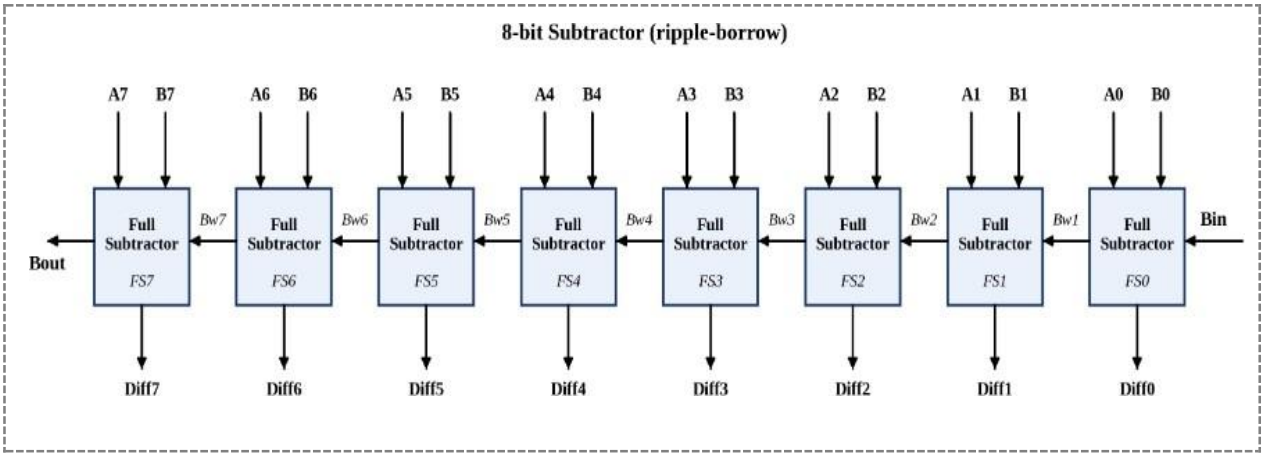


Figure 23: Block diagram of the 8-bit Subtractor (ripple-borrow structure).

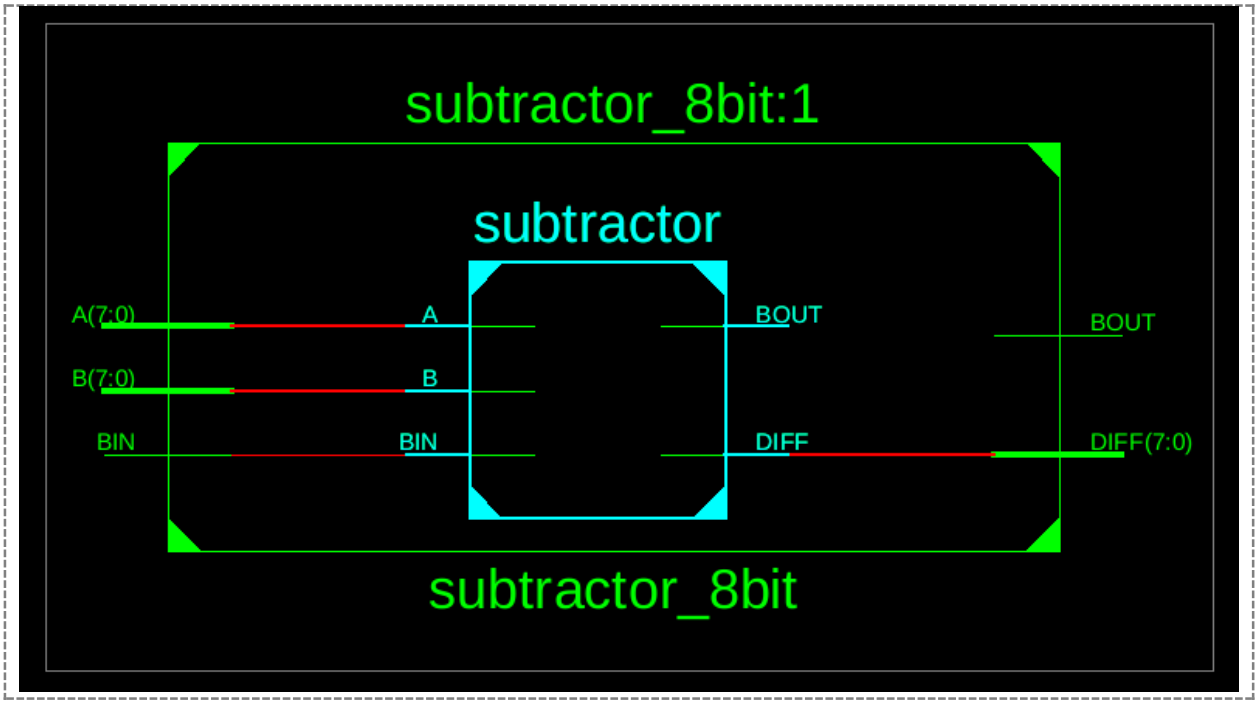


Figure 24: RTL schematic of the 8-bit Subtractor.

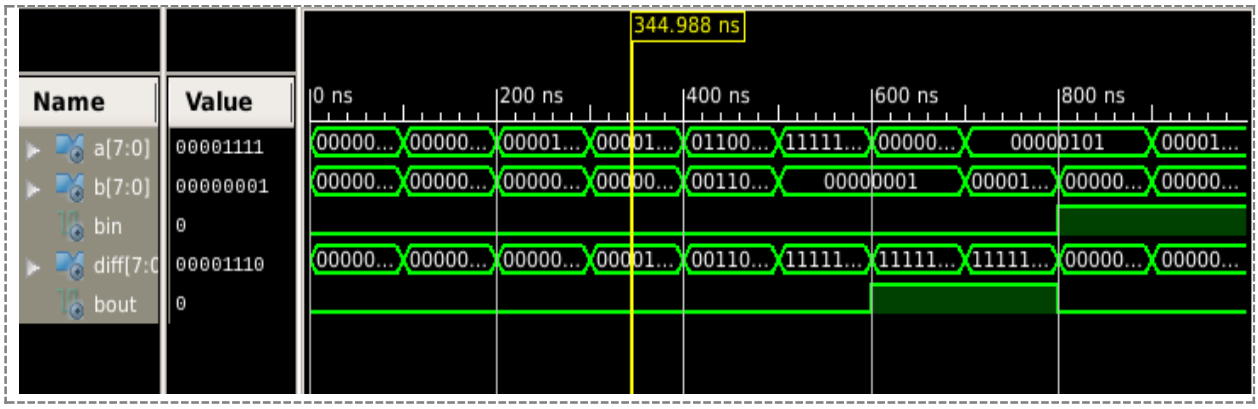


Figure 25: Simulation waveform of the 8-bit Subtractor.

12. 8-bit Multiplexer (MUX) Design

A multiplexer selects one of several input signals and forwards it to the output. In the ALU, an 8-bit MUX with two data inputs is used to choose between the output of the 8-bit Full Adder and the output of the 8-bit Subtractor. The select line decides which result is passed to the ALU output.

The Boolean expression of a 1-bit 2-to-1 MUX is:

$$Y = \overline{S} \cdot I0 + S \cdot I1$$

The 8-bit MUX applies this operation to each of the eight bit positions using the same select line S.

1-bit 2-to-1 MUX Truth Table

S	I0	I1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Block Diagram

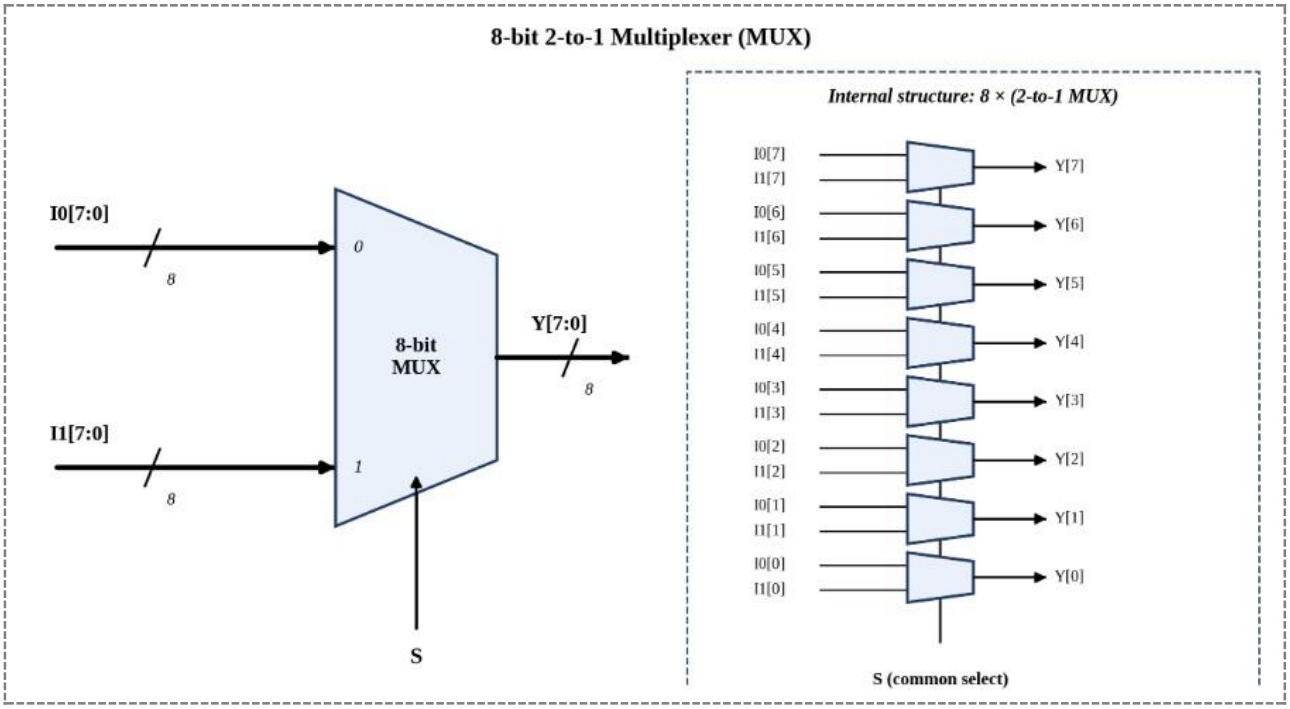


Figure 26: Block diagram of the 8-bit MUX.

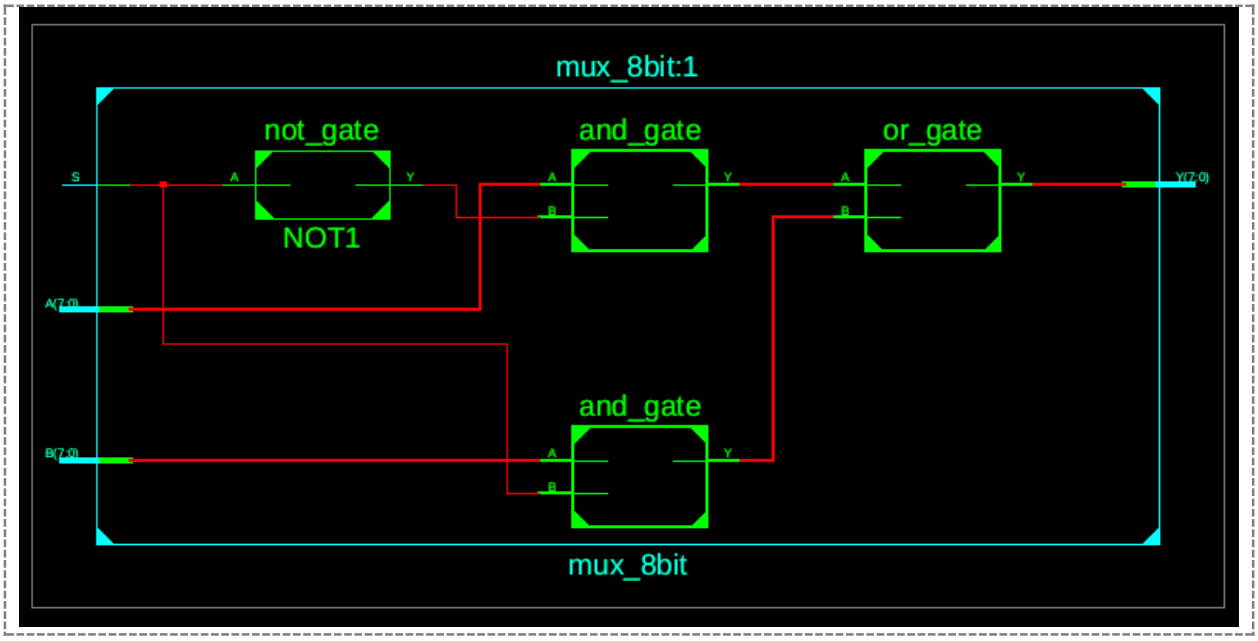


Figure 27: RTL schematic of the 8-bit MUX.

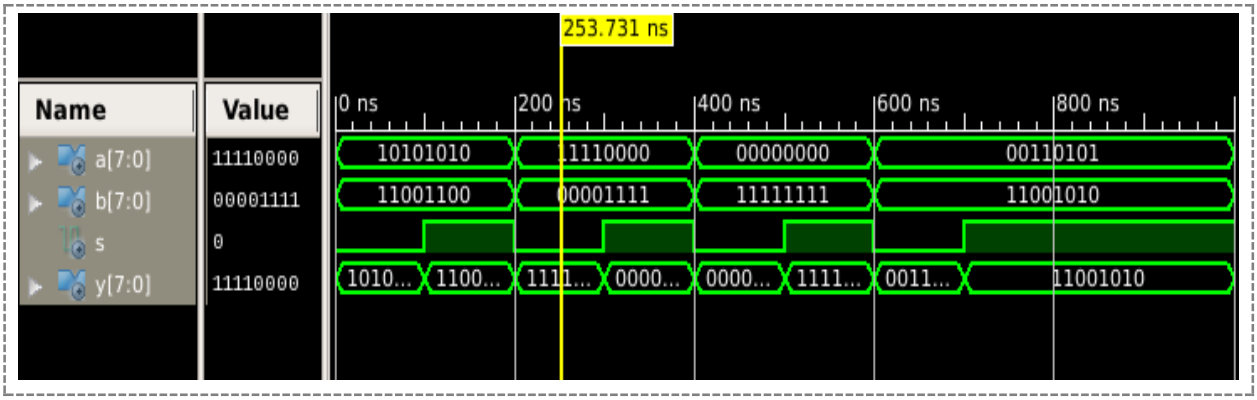


Figure 28: Simulation waveform of the 8-bit MUX.

13. ALU Design

The final 8-bit ALU was constructed by combining the 8-bit Full Adder, the 8-bit Subtractor, and the 8-bit MUX. The same operands A and B are applied to both the adder and the subtractor at the same time. The MUX then selects the required result according to the operation-select input.

Inputs

- A[7:0] – first 8-bit operand
- B[7:0] – second 8-bit operand
- Sel – operation select

Outputs

- Result[7:0] – 8-bit result of the selected operation
- Cout / Bout – carry output for addition or borrow output for subtraction

Operation Table

Sel	Operation	Block Selected by MUX	Result
0	Addition	8-bit Full Adder	A + B
1	Subtraction	8-bit Subtractor	A − B

Component Hierarchy and NAND Gate Usage

Component	Constructed Using	NAND Gates
NOT / AND / OR / XOR	NAND gates (Section 6)	1 / 2 / 3 / 4
Half Adder	1 XOR + 1 AND	6
Full Adder	2 Half Adders + 1 OR	15
8-bit Full Adder	8 Full Adders	120
Full Subtractor	2 XOR + 2 AND + 2 NOT + 1 OR	17
8-bit Subtractor	8 Full Subtractors	136
8-bit MUX	Gates built from NAND	—
8-bit ALU	8-bit Full Adder + 8-bit Subtractor + 8-bit MUX	—

ALU Block Diagram

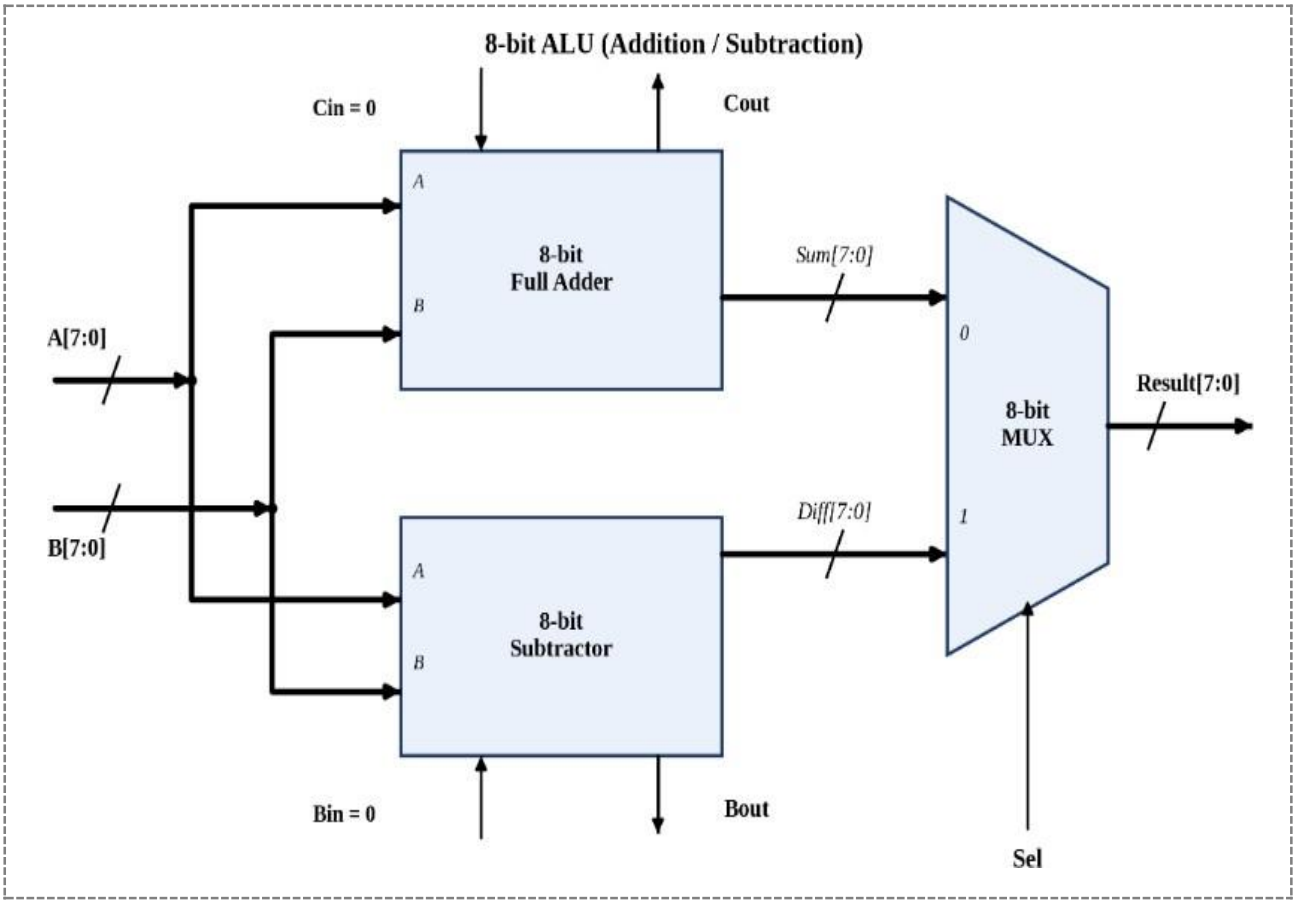


Figure 29: Block diagram of the 8-bit ALU.

14. RTL Schematic of the ALU

The final RTL schematic shows the hierarchical structure of the ALU, including the 8-bit Full Adder, the 8-bit Subtractor, and the 8-bit MUX.

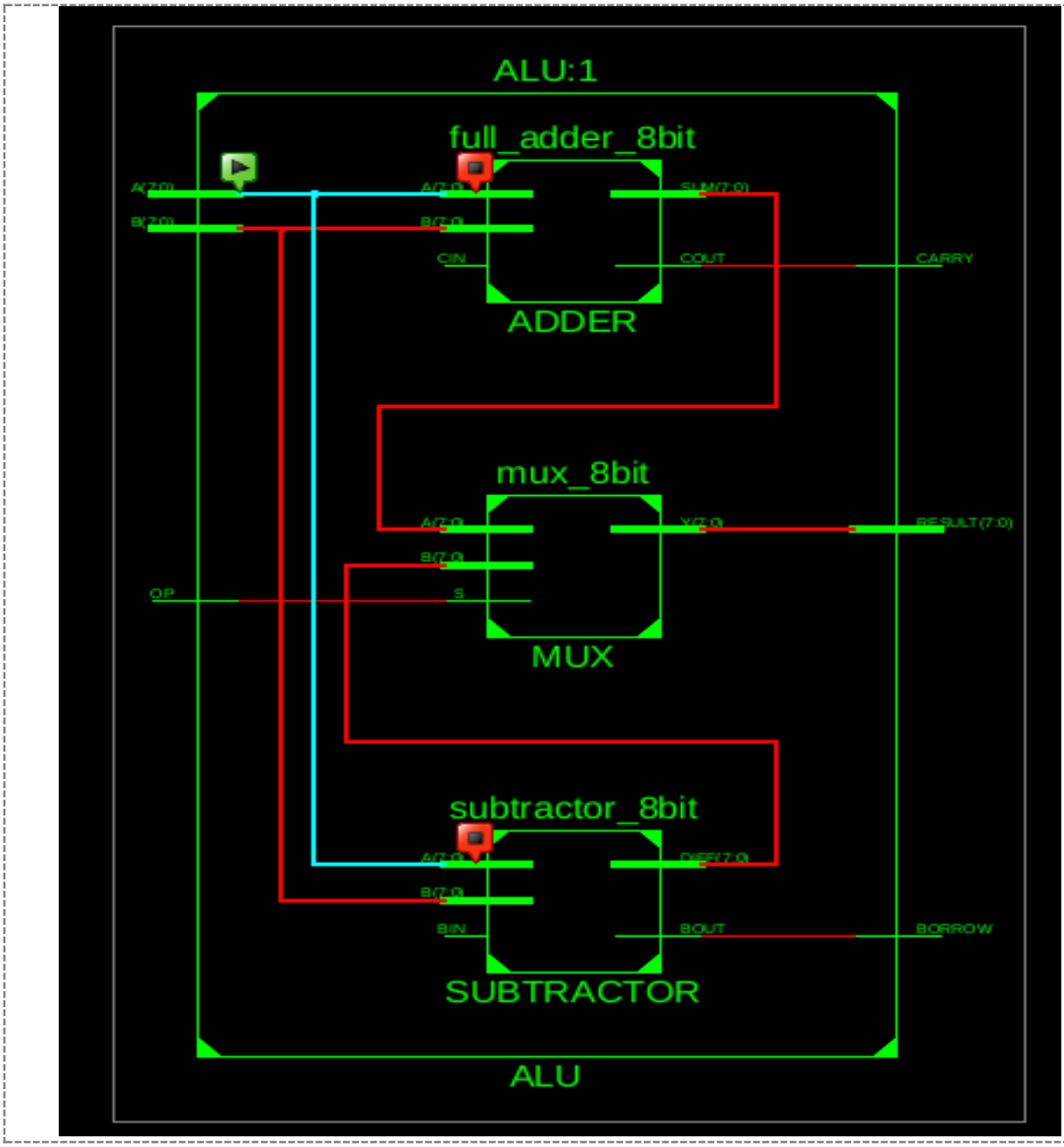


Figure 30: RTL schematic of the complete 8-bit ALU (top level).

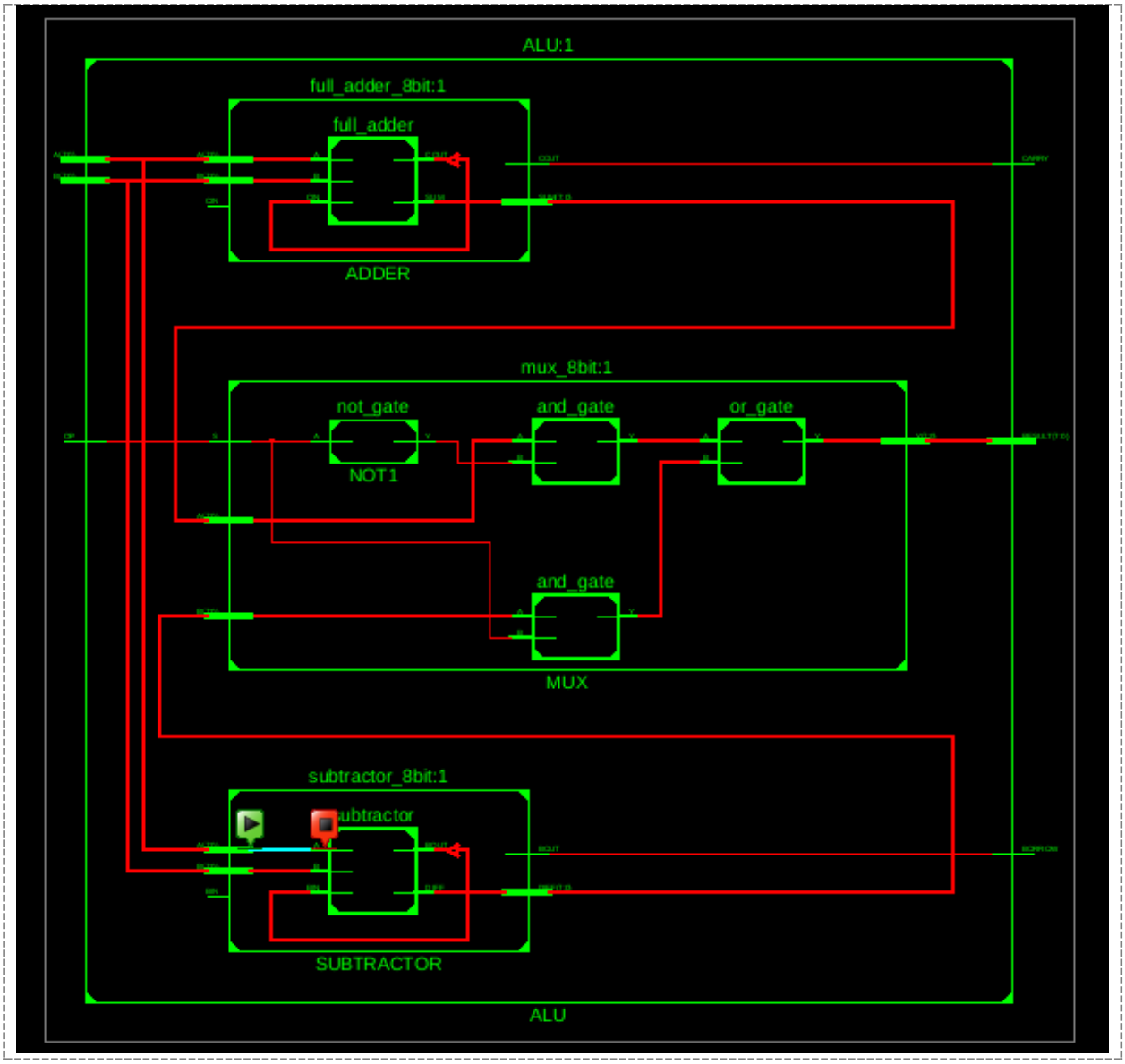


Figure 31: Expanded RTL schematic of the 8-bit ALU showing internal blocks.

15. Simulation

The complete ALU was simulated using the ISim simulator available in Xilinx ISE. The testbench applies different pairs of 8-bit operands for both operations (Sel = 0 for addition and Sel = 1 for subtraction) and observes the Result and Cout/Bout outputs.

ALU Testbench

The test cases applied in the testbench are given below:

No.	Sel	Operation	A	B	Result	Cout/ Bout
1	0	25 + 10 = 35	00011001	00001010	00100011	0
2	0	100 + 50 = 150	01100100	00110010	10010110	0
3	0	200 + 100 = 300	11001000	01100100	00101100	1
4	0	255 + 1 = 256	11111111	00000001	00000000	1
5	1	50 − 20 = 30	00110010	00010100	00011110	0
6	1	100 − 100 = 0	01100100	01100100	00000000	0
7	1	20 − 50 = -30	00010100	00110010	11100010	1
8	1	0 − 1 = -1	00000000	00000001	11111111	1

Each input combination is applied for a fixed time interval so that the corresponding outputs can be observed clearly.

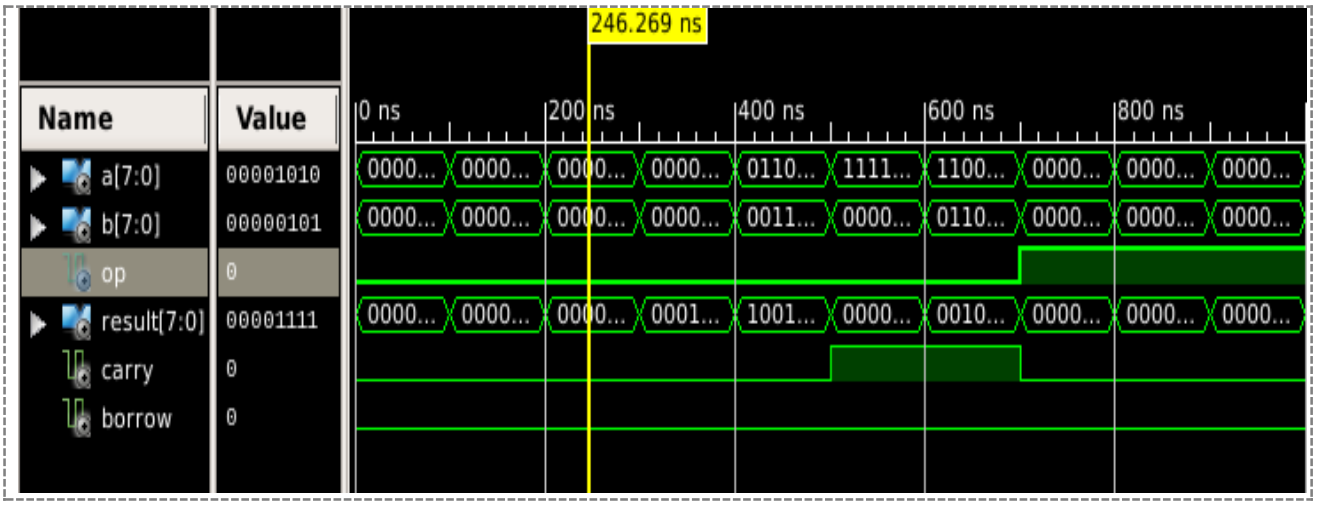


Figure 32: Simulation waveform of the ALU performing addition (OP = 0).

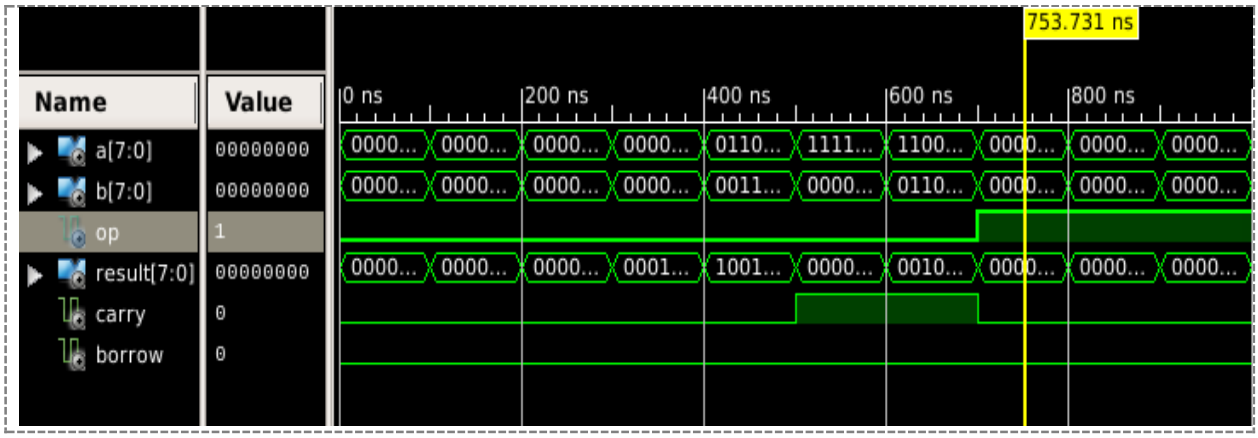


Figure 33: Simulation waveform of the ALU performing subtraction (OP = 1).

For the waveform screenshots, make sure the following signals are visible: A, B, Sel, Result, and Cout/Bout. It is preferable to display the buses in binary or decimal radix so that the values can be easily verified.

16. Simulation Result Analysis

The simulation verifies the functionality of the designed ALU. For example:

Case 1: Addition (Sel = 0)
A = 25 (00011001)
B = 10 (00001010)
Result = 35 (00100011)
Cout = 0

Case 2: Addition with carry out (Sel = 0)
A = 200 (11001000)
B = 100 (01100100)
The true sum is 300, which cannot be stored in 8 bits. Therefore:
Result = 44 (00101100)
Cout = 1

Case 3: Subtraction (Sel = 1)
A = 50 (00110010)
B = 20 (00010100)
Result = 30 (00011110)
Bout = 0

Case 4: Subtraction with borrow (Sel = 1)
A = 20 (00010100)

B = 50 (00110010)
Since A is smaller than B, the result is negative and appears in 2's complement form:
Result = 226 (11100010), which represents −30
Bout = 1

The simulation output agrees with the expected theoretical results for both addition and subtraction.

17. GitHub Repository

All VHDL source codes, testbenches, and related project files have been uploaded to a GitHub repository.

GitHub Repository Link:

https://github.com/rsf-rawnak/CSE450-VLSI-Labs/tree/main/ALU

18. Conclusion

In this experiment, a complete **8-bit ALU capable of addition and subtraction was designed and simulated using VHDL and Xilinx ISE**. The design followed a hierarchical structural approach.

Initially, a NAND gate was implemented using dataflow modeling. Since NAND is a universal logic gate, it was used to construct the NOT, AND, OR, and XOR gates. A Half Adder was built using XOR and AND gates, and a Full Adder was built using two Half Adders and an OR gate. Eight Full Adders were then connected to form an 8-bit Full Adder.

For subtraction, a Full Subtractor was built using two XOR gates, two AND gates, two NOT gates, and one OR gate, and eight of them were connected to form an 8-bit Subtractor. An 8-bit MUX was designed to select between the adder and subtractor outputs, and all these components were combined to build the final ALU.

The simulation results for the individual gates, adders, subtractors, MUX, and the complete ALU matched the expected truth tables and calculated values. Therefore, the design was successfully verified.

The experiment also demonstrated how **structural VHDL modeling, component instantiation, hierarchical design, and simulation** can be used to develop complex digital circuits from simpler reusable building blocks.